# SWIFTDROP

## Parcel Delivery & Courier Management System

Design Documentation: Use Cases, Data Model, Architecture, Integration Design, Key Flows and Justifications

| Item | Details |
|---|---|
| Module | Enterprise Application Development (Group Assignment, 20 marks) |
| Group number | 23 |
| Team size | 10 members |
| Submission | Design documentation report |
| Date | 29 October 2026 |

Design proposal. Architecture, controls and tests described here are intended behavior. No application implementation or test results were supplied for this review.

# Contents

Contents 2
List of Figures 4
1 Introduction 5
1.1 Background 5
1.2 Purpose of this document 5
1.3 Scope 5
1.4 Assumptions and constraints 5
1.5 Technology choices 6
2 Users, Roles and Access Rules 7
3 Use Case Modelling 8
3.1 Customer and recipient 8
3.2 Merchant 9
3.3 Courier 10
3.4 Dispatcher and admin 11
3.5 Use case specifications 12
4 Data Design 13
4.1 Entity overview 14
4.2 Design decisions 15
5 System Architecture 16
5.1 Options we considered 16
5.2 The chosen architecture 17
5.3 Inside the Core API 18
5.4 Why there is no separate BFF 18
5.5 Deployment view 19
6 Integration Design 20
6.1 Integration patterns used 20
6.2 Event flow 21
6.3 Event contract 22
6.4 Queues and routing keys 22
6.5 Reliability guarantees 23

# Contents (continued)

6.6 Real-time tracking 23
7 Key Flows 24
7.1 Creating a delivery 24
7.2 Merchant creates an order through the API 25
7.3 Courier status update triggering notifications 26
7.4 OAuth 2.0 login with Google 27
7.5 Parcel lifecycle 28
7.6 Dispatcher assigns a parcel 29
7.7 Webhook delivery and retry 30
8 Design Patterns and Principles 31
8.1 Patterns applied 31
8.2 Principles 32
9 Security Design 33
9.1 Authentication 33
9.2 Authorisation 33
9.3 Merchant API and webhooks 33
9.4 Personal data 34
9.5 Secrets and configuration 34
9.6 Threats and mitigations 34
10 API Design 35
10.1 Conventions 35
10.2 Main endpoints 35
10.3 Error format 36
11 Testing Approach 37
12 Bonus Features 38
12.1 Kubernetes deployment 38
12.2 AI delivery assistant 38
13 Conclusion 39
Appendix A: Contribution Sheet 40
Appendix B: Editing and Review Notes 41

# List of Figures

**Figure 1:** Customer and recipient use cases 8

**Figure 2:** Merchant use cases 9

**Figure 3:** Courier use cases 10

**Figure 4:** Dispatcher and admin use cases 11

**Figure 5:** Conceptual entity-relationship diagram 13

**Figure 6:** Layered view of the SwiftDrop architecture 17

**Figure 7:** Container deployment view (baseline and optional) 19

**Figure 8:** Event-driven integration through RabbitMQ 21

**Figure 9:** Sequence: customer creates a delivery 24

**Figure 10:** Sequence: merchant API order and webhook 25

**Figure 11:** Sequence: courier status update and fan-out 26

**Figure 12:** Sequence: Google sign-in with authorization code and PKCE 27

**Figure 13:** State diagram: parcel lifecycle 28

**Figure 14:** Activity diagram: assignment and reassignment 29

**Figure 15:** Activity diagram: webhook delivery with retry 30

**Figure 16:** Optional AI assistant through authenticated API tools 38

# 1 Introduction

## 1.1 Background

SwiftDrop is a courier company delivering parcels across Sri Lanka. Bookings have come in over the phone and been tracked in shared spreadsheets. Customers call the hotline for updates, dispatchers juggle courier assignments from memory, and merchant order lists are retyped by staff, causing delays and address or phone errors.

The proposed platform lets customers book and track parcels, merchants submit orders directly from their e-stores, dispatchers see unassigned parcels and courier workloads, and couriers update progress from their phones.

## 1.2 Purpose of this document

This report records the system design and the reasons behind it: use cases, data model, architecture, integration contracts, key sequences and decision flows, security and testing. It gives evaluators and the ten-person team a shared design before implementation.

## 1.3 Scope

Baseline: customer, merchant, courier, dispatcher and admin functions; JWT authentication with Google sign-in; a scoped merchant API; event-driven email, mock SMS and signed webhooks; live tracking; and local Docker Compose deployment. Kubernetes/Helm and the AI assistant are optional stretch features, started only after the baseline is demoable.

Out of scope: online card payments, route optimisation, warehouse/hub inventory, native mobile apps and tracking the physical return-to-sender journey. Fees are recorded and settled offline. The courier UI is a mobile-first PWA.

## 1.4 Assumptions and constraints

• Prices use integer Sri Lankan Rupee cents. Origin zone, destination zone, size and weight determine the fee. Zones follow district groupings.

• Couriers use mid-range Android phones on patchy mobile data. Offline updates need conflict handling; local queueing alone cannot guarantee acceptance.

• The team has roughly one semester and favours familiar TypeScript tooling. The intended local start command is docker compose up; deployment files do not exist yet.

## 1.5 Technology choices

The familiar TypeScript stack is retained. Exact dependency versions and image digests must be pinned and checked together before implementation.

| Concern | Choice | Why it fits |
|---|---|---|
| Frontend | React 18, Vite, TypeScript, Tailwind CSS | Shared components for five account roles and a public tracking page. |
| Backend | Node.js 24 LTS, Express, TypeScript | Supported runtime line and a familiar HTTP/OAuth/AMQP ecosystem. |
| Validation | Shared Zod schemas | Consistent browser and API rules; the API remains the trust boundary. |
| Database | PostgreSQL 16, Prisma | Relational constraints and atomic parcel/history/outbox transactions. |
| Broker | RabbitMQ topic exchange; supported release to be pinned | Broker-assisted delays and fan-out. Application code still implements retry policy, bookkeeping and safe transfers. |
| Real-time | Socket.IO, Redis adapter | Fallback to long-polling and cross-replica room delivery. |
| Object storage | MinIO, private S3-compatible buckets | Proof photos and receipts outside the database; authorised signed URLs. |
| Authentication | RS256 JWT, Google OIDC, OAuth client credentials | User access, social sign-in and scoped machine access. |
| Testing | Jest, Supertest, Testcontainers; Vitest and React Testing Library | Planned checks against real database/broker dependencies and UI component tests. |
| Deployment | Docker Compose baseline; Kubernetes/Helm optional | Reproducible local environment; reuse images if stretch work proceeds. |

Runtime correction: Node.js 20 is EOL. Node.js 24 is an LTS line on the official release table as reviewed on 29 October 2026. Use a current security patch within the chosen line and verify dependency compatibility before pinning. Source: Node.js release table.

## 2 Users, Roles and Access Rules

Five authenticated account roles are Customer, Merchant, Courier, Dispatcher and Admin. Each account has one role. The guest Recipient has no account. A merchant e-store is a separate machine actor using scoped client credentials, not a sixth human role. Admin inherits dispatcher operations and additionally controls configuration and replay.

| Actor | Goals and authentication |
|---|---|
| Customer | Book, track, cancel before pickup, download receipt; email/password or Google. |
| Recipient (guest) | Public tracking of one parcel by tracking number; no login. |
| Merchant staff / e-store | Staff dashboard via user JWT; approved e-store via scoped machine JWT. |
| Courier | Assigned jobs, pickup, status and proof; admin-provisioned password account. |
| Dispatcher | Operational parcel creation/status, assignment, workload and reports; password account. |
| Admin | Dispatcher operations plus users, merchant approval, zones, pricing and webhook replay. |

*Table 3: Role-based access matrix*

| Capability | Customer | Merchant | Courier | Dispatcher | Admin |
|---|---|---|---|---|---|
| Create parcel | own | own API | - | yes | yes |
| View parcel/history | own | own | assigned | all | all |
| Update status | - | - | assigned | yes | yes |
| Assign / reassign | - | - | - | yes | yes |
| Cancel before pickup | own | own | - | yes | yes |
| View reports | - | own orders | - | yes | yes |
| Manage API/webhook | - | own | - | - | yes |
| Approve merchants; users/zones/pricing | - | - | - | - | yes |
| Replay failed webhook | - | - | - | - | yes |

Every protected request checks role/scope and record ownership in the database query. Ownership-scoped misses return 404; known route-level role/scope denials return 403. Public responses contain only status, time and town. These are required controls, not verified implementation.

# 3 Use Case Modelling

Use case diagrams show user goals inside the SwiftDrop boundary and external actors outside it. Dashed include arrows point to required behavior; dashed extend arrows point from an optional extension to its base goal. Internal brokers and workers belong in architecture diagrams.

## 3.1 Customer and recipient

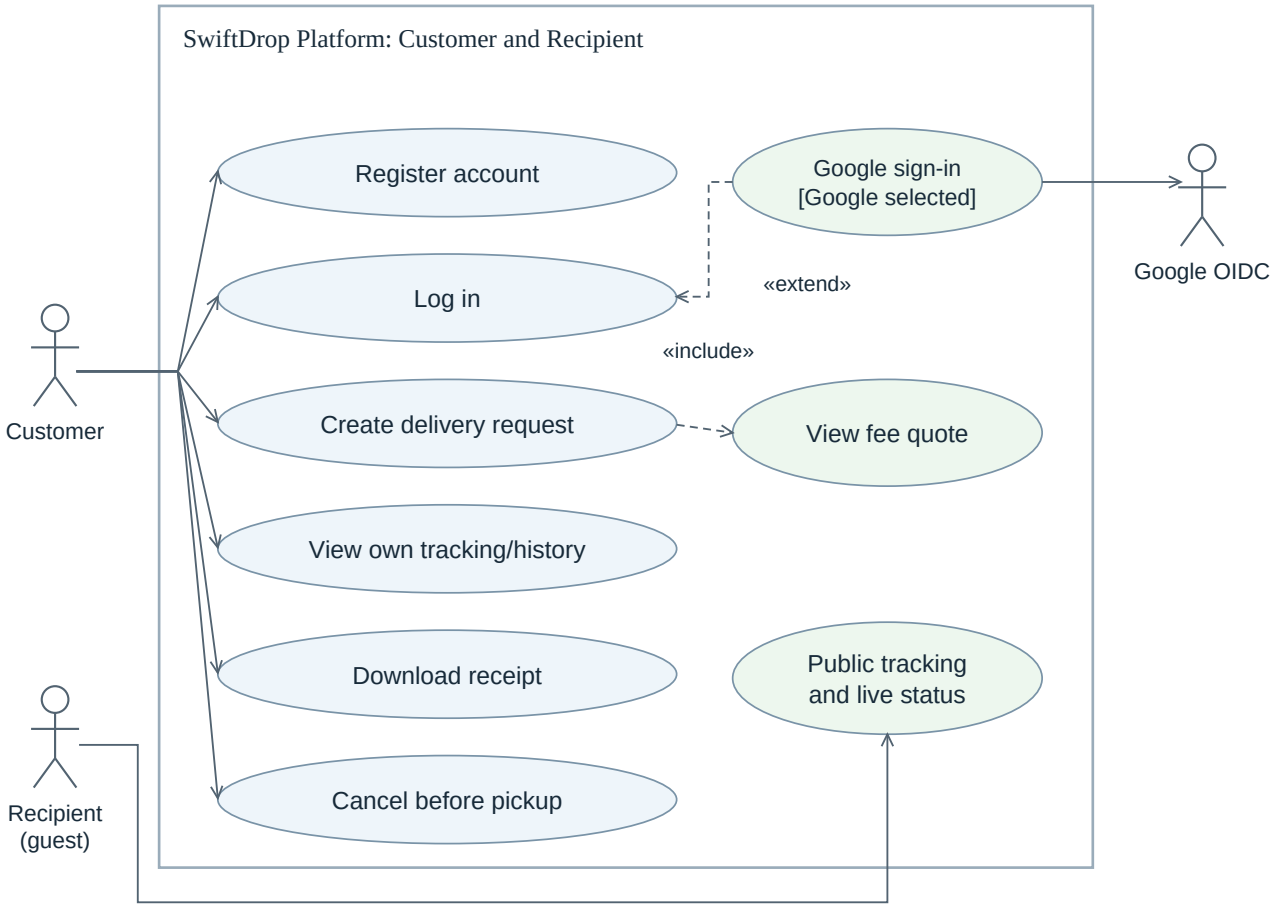



*Figure 1: Customer and recipient use cases*

Booking includes viewing a fee quote before confirmation. Google sign-in is an alternative login flow, shown as an extension only when the customer selects Google. Public tracking needs no account and exposes status, time and town only. Cancellation belongs to the owner and is allowed only before pickup.

Booking confirmation and status alerts follow committed business events. Receipt downloads and tracking-page visits do not send notification events. Email and mock SMS providers are external integrations; the Notification Service itself is internal.

## 3.2 Merchant

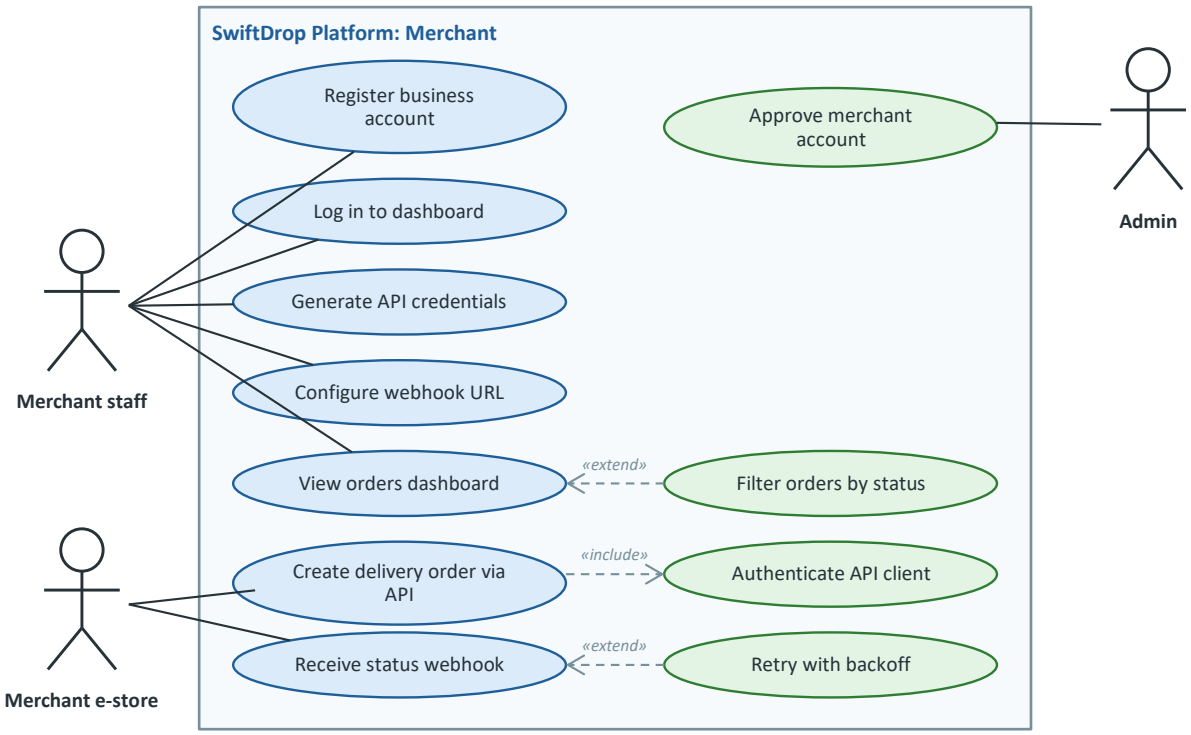



*Figure 2: Merchant use cases*

We modelled the merchant as two actors. **Merchant staff** are people using the dashboard. The **merchant e-store** is their software calling our API. The distinction matters for security: staff log in with a password and get a user JWT, while the e-store authenticates with client credentials and gets a token limited to order scopes. Registration needs an administrator's approval before the merchant can create API credentials, which stops anyone from signing up and flooding the system with fake orders.

## 3.3 Courier

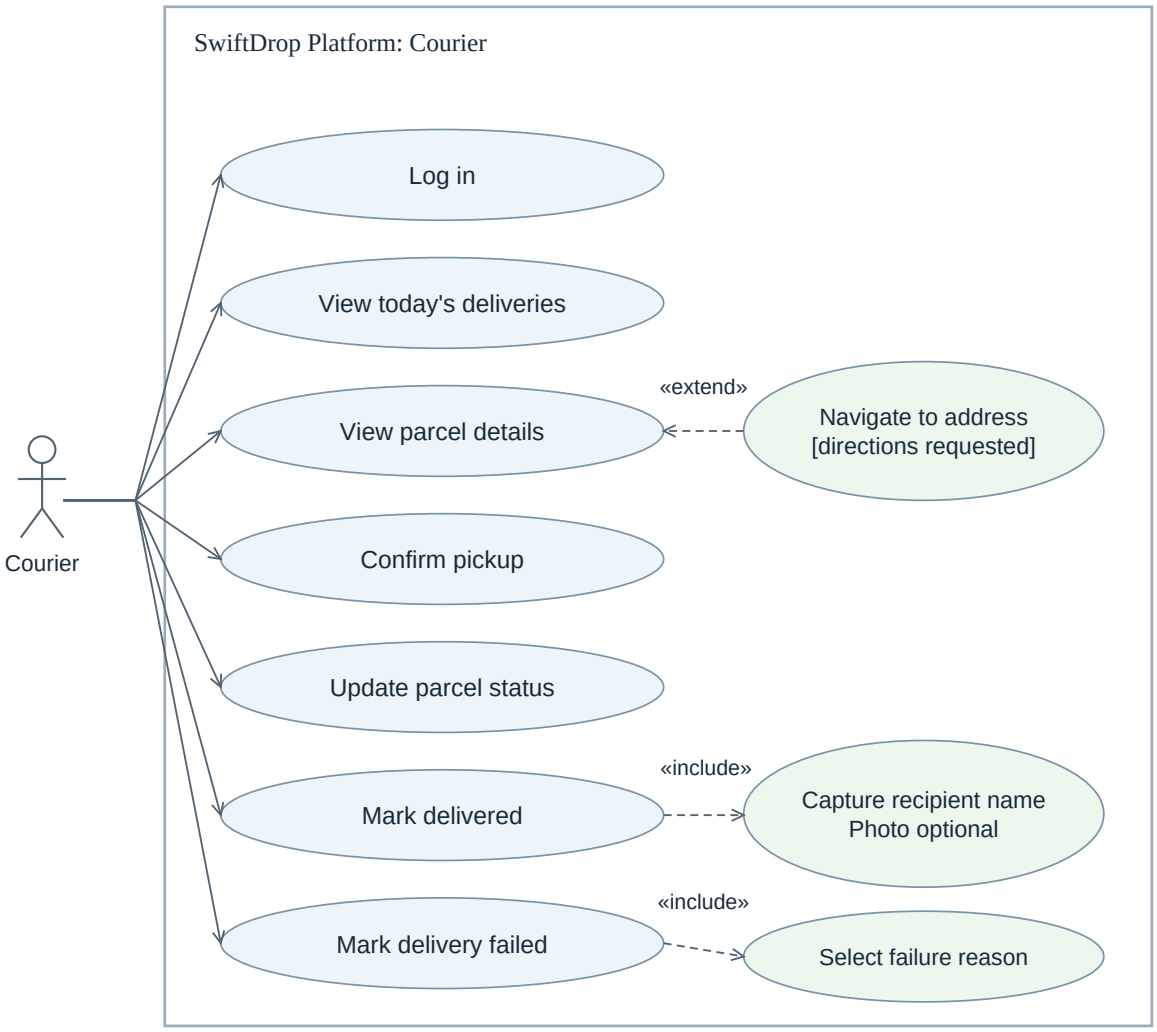



*Figure 3: Courier use cases*

Couriers operate only on parcels currently assigned to them. Navigation extends parcel details when directions are requested; its arrow points toward the base use case. Internal event publication is shown in the integration and status sequence diagrams.

Delivered requires the recipient name. A photo is optional and is stored as a private object. Failed requires a reason from a fixed list, for example recipient unavailable or address not found. The API validates the transition, current assignment, parcel version and proof before committing.

Offline replay retains the operation key and expected version. A stale, reassigned or invalid update is held for user review; the PWA must never silently overwrite newer server state. See Section 7.3.

## 3.4 Dispatcher and admin

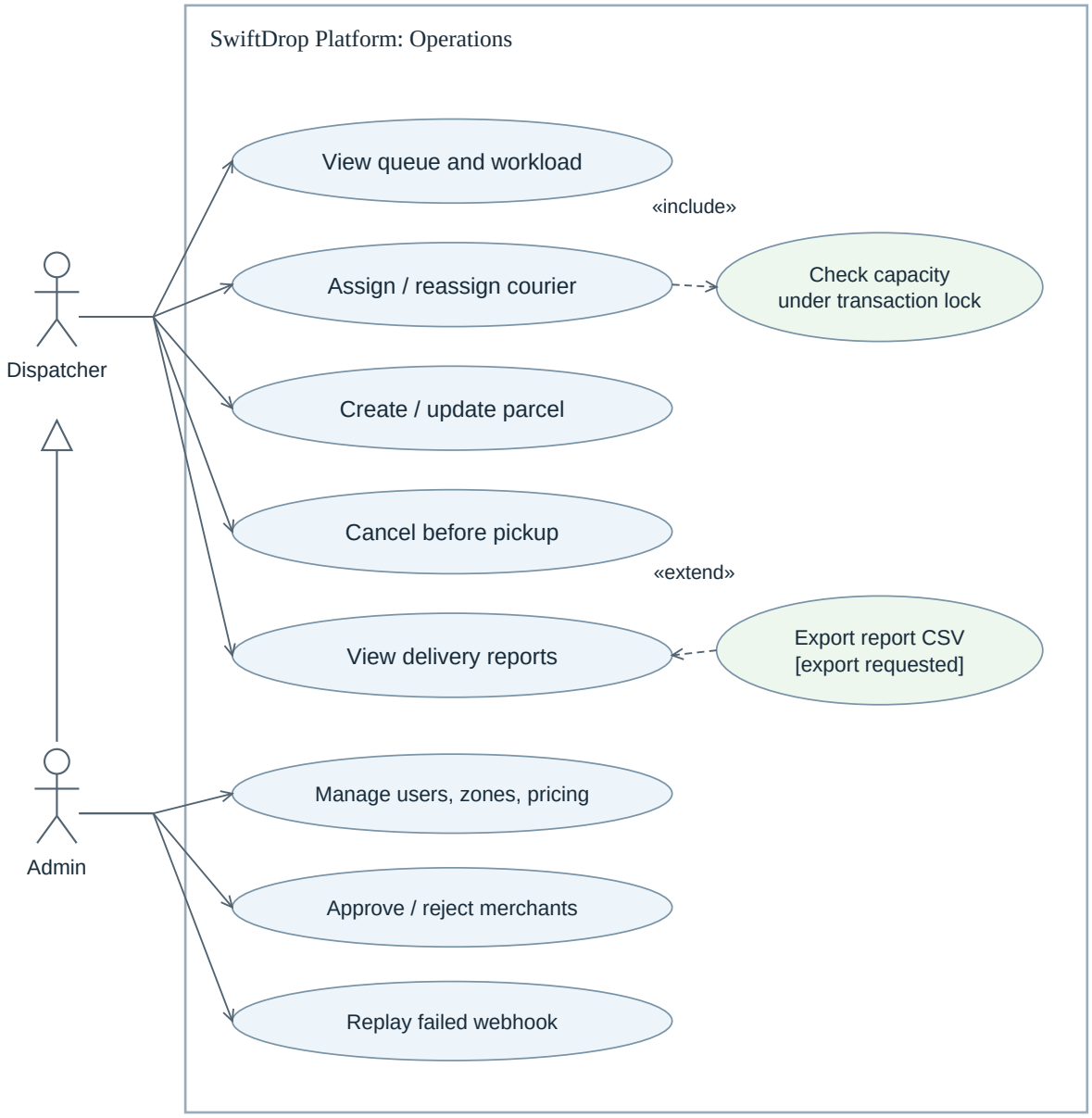



*Figure 4: Dispatcher and admin use cases (Admin inherits Dispatcher)*

Assignment includes the capacity check. Reassignment is the same operational goal with an existing courier, so it uses a branch in the assignment flow rather than a separate permission. The transaction releases old capacity and reserves new capacity together.

Admins inherit all dispatcher operations, including parcel creation, status updates and assignment. Only Admin may replay failed webhooks or change configuration. Dispatchers may inspect operational failures but do not receive replay permission.

## 3.5 Use case specifications

*Table 4: Create delivery request*

| Field | UC-C03 Create delivery request |
|---|---|
| Primary actor | Customer (staff may create operational parcels under the same validation rules). |
| Preconditions | Authenticated active account. Email verification is not a baseline booking precondition; no unspecified verification workflow is implied. |
| Main flow | 1. Enter pickup/drop-off addresses, recipient name/phone, size and weight.<br>2. Validate input and resolve both zones.<br>3. Persist an owner-bound quote valid for 15 minutes; display the fee.<br>4. Confirm quoteId with an Idempotency-Key.<br>5. Atomically create parcel, first history row, saved idempotency response and outbox event; commit.<br>6. Return tracking number. The relay later publishes parcel.created; the notification worker sends the confirmation email. |
| Alternatives | Unserved zone: validation error. Expired quote: recalculate and request confirmation. Same key/body: return the saved response; same key/different body: 409. Database failure: roll back. Broker outage: committed outbox waits for retry. |
| Postconditions | Parcel is CREATED, has no courier and appears in the unassigned queue; event delivery is at least once. |

*Table 5: Update parcel status*

| Field | UC-K05 Courier update |
|---|---|
| Preconditions | Active Courier account; parcel currently assigned to the caller. |
| Main flow | 1. Send status, expectedVersion and stable operation key.<br>2. DELIVERED: recipient name required, photo optional; FAILED: fixed-list reason required.<br>3. Check ownership, legal transition, version and private proof object.<br>4. Commit parcel/version, history, idempotency response and outbox atomically.<br>5. Return success; relay and consumers update subscribers asynchronously. |
| Alternatives | Foreign parcel: 404. Role denial: 403. Stale version/illegal transition: 409. Offline: queue for replay; same key reuses the saved response, conflicts need review. Failed photo upload blocks sending a photo reference; retry upload or omit the optional photo. |
| Postconditions | State/history/outbox recorded atomically; stable eventId supports duplicate suppression. No exactly-once broker or external-send guarantee. |

# 4 Data Design

Figure 5 is a conceptual relationship view, not a migration-ready schema. Sections 4.1-4.2 supply the persistence contracts omitted from the compact diagram. PK identifies a primary key; FK identifies a foreign key; optional ownership is explicit.

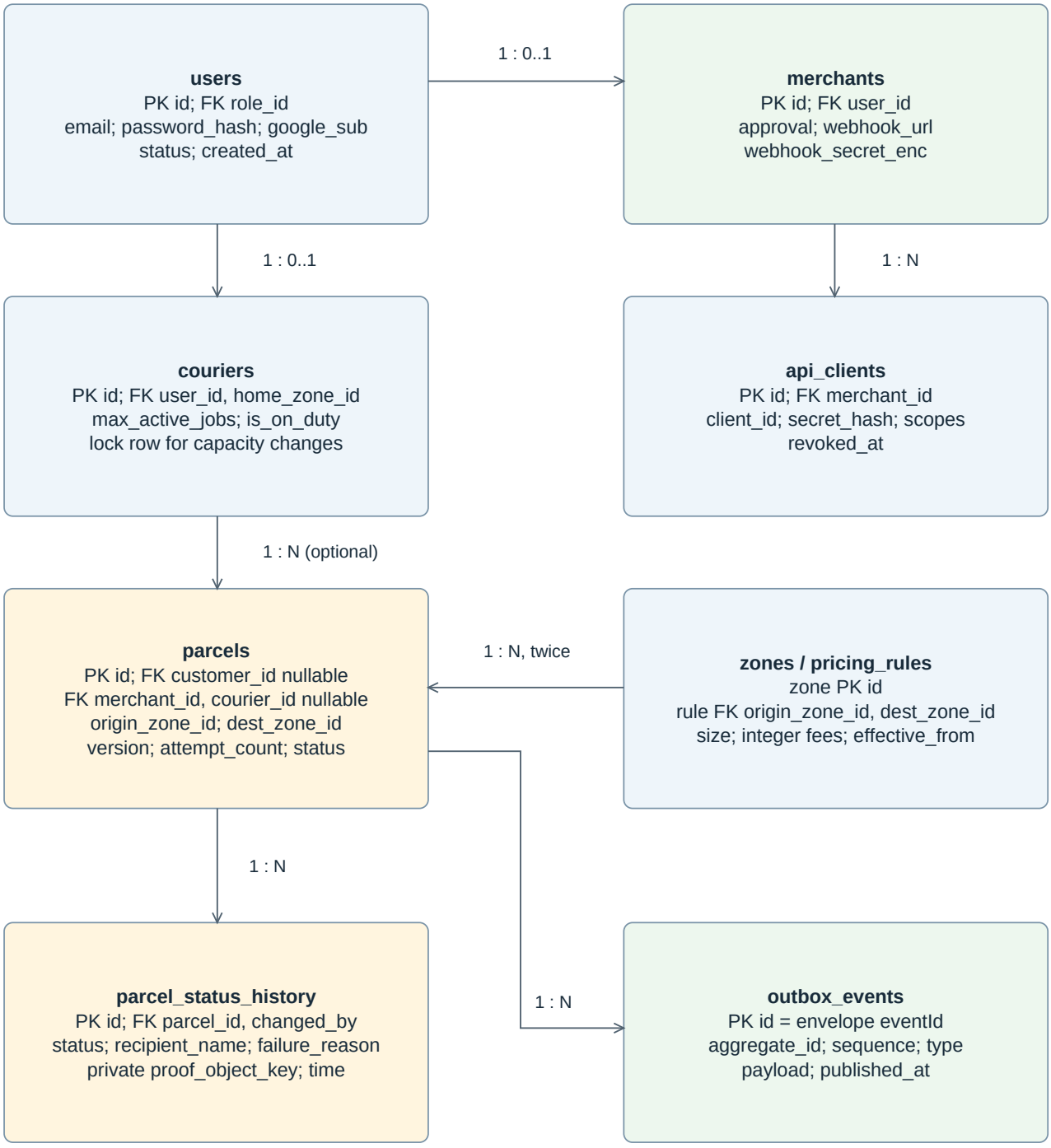



*Figure 5: Conceptual entity-relationship diagram (selected entities)*

Customer owns zero or more parcels; customer_id is nullable for machine-created merchant orders. Merchant owns its orders through merchant_id; either customer_id or merchant_id must be present. Both parcel zone FKs and both pricing-rule zone FKs reference zones.

## 4.1 Entity overview

*Table 6: Required persistence contracts (conceptual)*

| Structure | Required fields / invariants |
|---|---|
| Identity and merchant data | users/roles, merchants and api_clients as in Figure 5; one account role; hashed client secrets, encrypted webhook signing secrets; approved merchant required for machine orders. |
| parcels | Unique tracking_no; nullable customer_id and merchant_id with at least one owner; nullable courier_id; pickup/drop-off addresses and towns; recipient name/phone; size/weight; fee_cents; origin_zone_id and dest_zone_id; status, version, attempt_count (0..3), scheduled date and timestamps. |
| parcel_status_history | Append-only parcel_id, status, changed_by, changed_at, recipient_name when delivered, failure_reason when failed, note, nullable private proof_object_key. No permanent public proof URL. Assignment/reassignment entries also record old/new courier. |
| quotes | id, owner type/id, normalised input or input hash, zone IDs, pricing-rule version, fee_cents, expires_at, consumed_parcel_id. Confirmation atomically checks owner, expiry and single consumption. |
| refresh_tokens | id, user_id, family_id, token_hash, expires_at, used_at, revoked_at, replaced_by. Rotation consumes once; reuse revokes the family atomically. |
| idempotency_records | Unique (principal, operation, key); request_hash; response status/body; created_at/expires_at. Concurrent same-key requests serialize; different-body reuse returns 409. Bind created parcels/updates to the record. |
| outbox_events | Stable id is envelope eventId; aggregate_id, aggregate sequence, event_type, schema version, occurred_at, PII-minimised payload, created_at, published_at. Relay preserves eventId on every retry. |
| processed_events / notifications | Unique (consumer, eventId); processing lease/state and completion time. Notifications add eventId, channel, template, masked recipient, status, sent_at and provider idempotency key when supported. |
| webhook_deliveries / attempts | One delivery per (merchant_id,eventId,replay_run). Store attempt number 0..6, next_retry_at, state and a row per completed HTTP attempt: code/error, latency, timestamp. Unique attempt keys; durable worker leases prevent concurrent sends. |

This contract still needs concrete migrations, FK/delete policies, indexes, cleanup/retention jobs and schema validation. It must not be described as a complete implemented database.

## 4.2 Design decisions

• **Append-only history.** Every committed status change has an actor and timestamp. Never rewrite past events; record corrections as new authorised entries. Current status and the history entry are committed in the same transaction.

• **Integer money and durable quotes.** Store fee_cents, base_fee_cents and per_kg_cents as integers. Bind a quote to its principal and inputs, expire it after 15 minutes and consume it once, so later price changes cannot alter an accepted charge.

• **Random tracking numbers.** Use SD- plus eight random characters from an unambiguous alphabet and a check character. Enforce uniqueness and retry a collision. The public endpoint is rate-limited and contains no private contact data.

• **Versioned parcel writes.** Updates require expectedVersion and increment version. A zero-row update means conflict. This protects one parcel but does not protect capacity across different parcels.

• **Courier capacity locking.** Every operation that changes assignment, active status or duty/capacity takes the affected courier row locks in sorted ID order before locking the parcel. Recheck active workload inside the transaction. Active means assigned and not DELIVERED, CANCELLED or terminal FAILED after attempt 3; retryable FAILED still reserves a slot. Reassignment and cancellation use the same protocol.

• **Delivery attempts.** attempt_count starts at 0 and increments when entering OUT_FOR_DELIVERY. Failed attempt 1 or 2 may be scheduled again by Dispatcher/Admin; attempt 3 is final. The final FAILED state releases active capacity; the physical return journey is out of scope.

• **Indexes follow bounded queries.** Plan indexes for tracking_no, (courier_id,current_status), (merchant_id,created_at), origin_zone_id and (parcel_id,changed_at). Lists use cursor pagination; workload queries consider only active parcels.

• **Idempotency and cleanup.** Store principal-bound operation keys and replies for at least the supported offline/retry window (initial target: 7 days). After expiry, clients must reconcile server state before retrying creation. Retain processed-event keys for at least the broker/DLQ and replay windows; deduplication storage is not a proof of exactly-once external delivery.

> Lock order is shared by all capacity-changing paths. If the observed courier changes before locks are acquired, roll back and retry from a fresh snapshot. This is a design requirement to verify under concurrent integration tests.

# 5 System Architecture

## 5.1 Options we considered

The modular monolith with independent workers remains the chosen architecture. The correction concerns its justification, not a wholesale redesign.

| Option | Strengths | Trade-offs / decision |
|---|---|---|
| Layered monolith | Simple build, tests and deployment; background jobs are possible. | Layering does not force external calls into HTTP handlers. Keeping workers in one deployment ties failure recovery and scaling together. Not chosen for outbound integrations. |
| Full microservices | Independent deployment and scaling; clear service ownership. | Ten students would take on distributed transactions, deployment and cross-service diagnosis before baseline features are ready. Rejected at this scale. |
| Modular monolith with event-driven workers | One database transaction for related business changes; independent outbound retries and consumer scaling. | Core modules share one release and need enforced boundaries. Chosen: simple transactional core, isolated slow integrations. |

Parcel creation, pricing, assignment and status changes are closely linked and benefit from sharing a transactional Core API. Email, SMS and merchant HTTP endpoints are slow or unreliable, so workers handle them after commit. An outage in those integrations does not block a courier update.

The baseline processes are Core API with Socket.IO, Outbox Relay, Notification Service and Webhook Dispatcher. The AI Assistant Service is a separate optional process. Redis and the broker are infrastructure dependencies; neither is an external actor in use case models.

> One PostgreSQL database is the system of record. The outbox closes the database-to-broker recording gap, while stable event identifiers and consumer state handle redelivery. It does not make database and broker publication one distributed transaction.

## 5.2 The chosen architecture

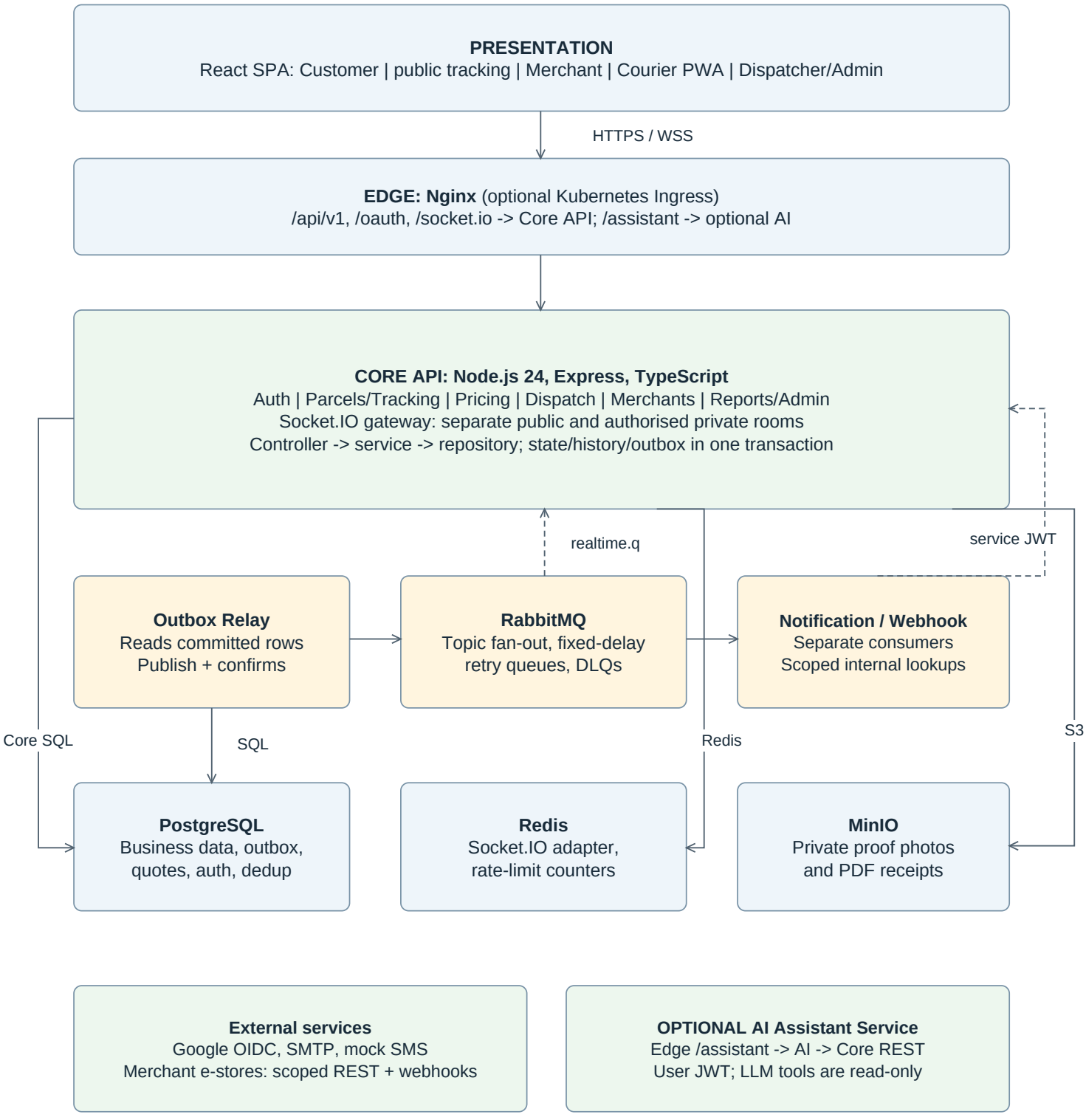



*Figure 6: Layered view of the SwiftDrop architecture*

The edge terminates TLS, serves the SPA and applies rate limits. Core API accesses PostgreSQL, Redis and private MinIO objects. Outbound workers use restricted service identities for necessary contact/configuration lookups; they do not read arbitrary core tables.

## 5.3 Inside the Core API

Each of the six modules owns its tables and exposes a small service interface. Dispatch calls PricingService.quote() when it needs a fee; it must not import another module's repositories. Planned ESLint import rules will enforce these boundaries in CI. No existing build enforcement is claimed.

Protected requests pass through authentication, role/scope checks and boundary validation, then a thin controller, a domain service and a Prisma repository. Controllers translate HTTP; services hold rules and do not touch req or res. The shared kernel contains validation, money, event envelope and error/logging utilities.

### Worker-to-API access

Notification and webhook workers receive short-lived service JWTs with separate audiences and narrow scopes, such as notifications:lookup and webhooks:config. Internal lookup endpoints are reachable only on the private network, require the service identity, and bind access to the referenced event and parcel/merchant. They return only the contact or signing material needed for that task; secrets and payloads are never logged.

Workers may persist their own delivery, attempt and processed-event state. The Outbox Relay alone receives a database role restricted to claiming unpublished outbox rows and updating publication metadata. User JWTs are not reused for background processing.

## 5.4 Why there is no separate BFF

The Core API already serves audience-specific routes: shared /api/v1/parcels plus /api/v1/merchant, /api/v1/courier and /api/v1/admin. These handlers return payloads suited to each screen, including a compact courier list for mobile data. A separate backend-for-frontend would add a deployable and a network hop without a demonstrated benefit at this scale.

Role-specific routes still use common domain services and ownership checks. Presentation grouping does not bypass authorisation. Public tracking and socket payloads use an explicit public projection rather than a full parcel serialisation.

> The six modules share one release; independent worker processes provide outbound failure isolation, retry and scaling. The optional assistant uses normal authorised APIs and does not gain direct database access.

## 5.5 Deployment view

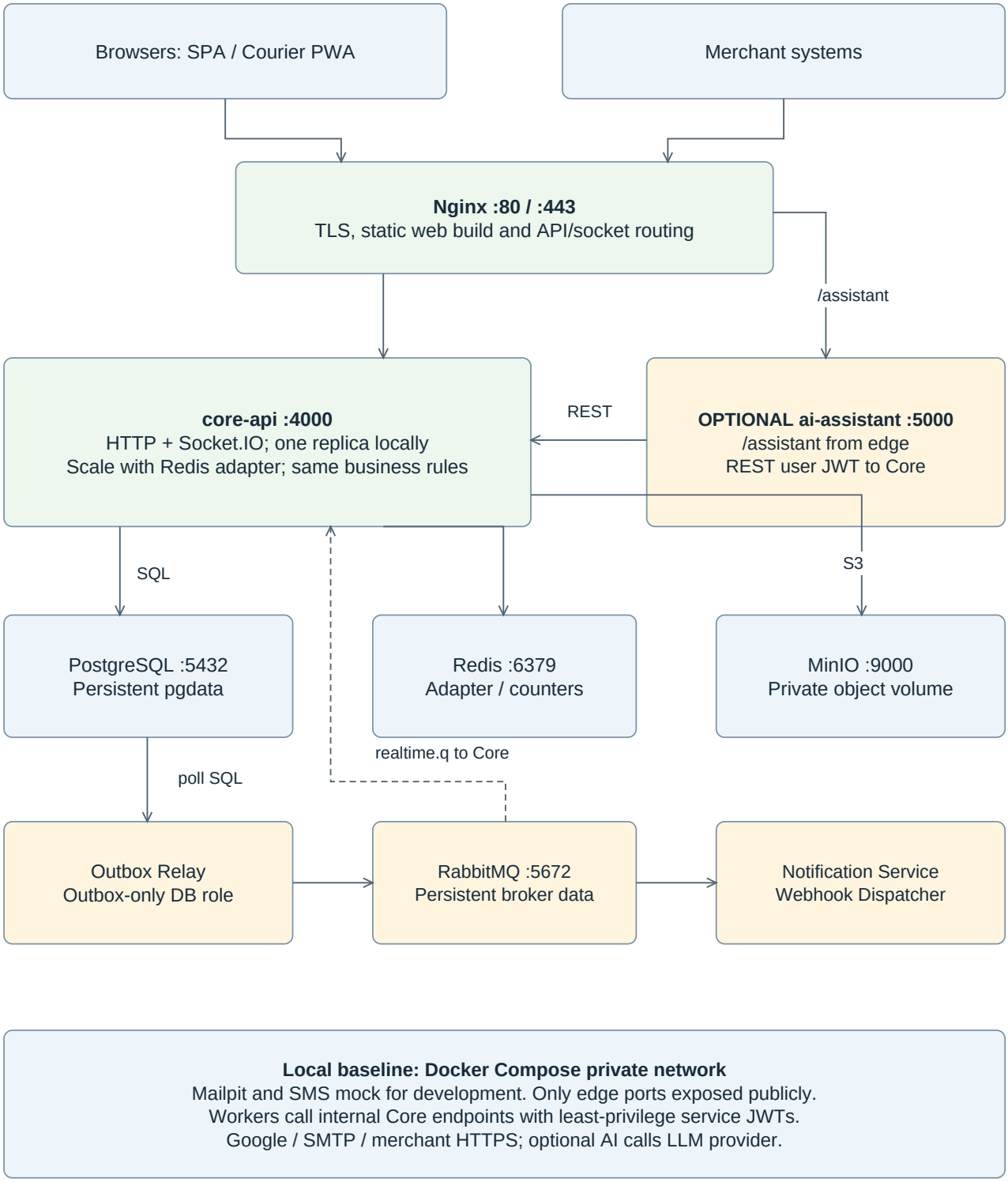



*Figure 7: Container deployment view (baseline and optional)*

Compose is the baseline. Kubernetes/Helm is stretch: Deployments for stateless processes, StatefulSets/PVCs for stateful services, Ingress/TLS, ConfigMaps and Secrets. Use multi-stage images and health probes. No Compose or Helm files have been implemented or verified.

# 6 Integration Design

## 6.1 Integration patterns used

Choose the pattern for the interaction: synchronous requests for decisions and asynchronous work for independent integrations. These are proposed contracts.

*Table 8: Integration patterns and justification*

| Pattern | Where used | Why it fits |
|---|---|---|
| Request / reply REST | SPA and merchant orders; optional assistant tools | The caller needs a quote, tracking number or validation response immediately. |
| Publish / subscribe | Committed parcel events to notification, webhook and realtime consumers | Consumers can run and fail independently; the producer does not wait for external work. |
| Transactional outbox | Parcel/history and outbox row in one PostgreSQL transaction | Atomic durable recording; relay retries broker publication after commit. At least once, not exactly once. |
| Signed webhook | Webhook Dispatcher to merchant HTTPS endpoint | Pushes changes without merchant polling; HMAC and timestamp authenticate each delivery attempt. |
| Retry with backoff and DLQ | Webhook delivery and bounded notification retries | Persistent bookkeeping and broker-assisted delays cover transient outages, then expose failures for inspection. |
| Server push | Socket.IO gateway to tracking clients | Immediate UI feedback with long-poll fallback and reconnect snapshot; separate public/private rooms protect data. |
| OAuth 2.0 / OpenID Connect | Google code flow with PKCE; merchant client credentials | Standard user and machine authentication with separate identities and scopes. |

The baseline has three async workers: Outbox Relay, Notification Service and Webhook Dispatcher. Realtime consumption stays inside Core API. An optional AI service is not counted as a core worker.

## 6.2 Event flow

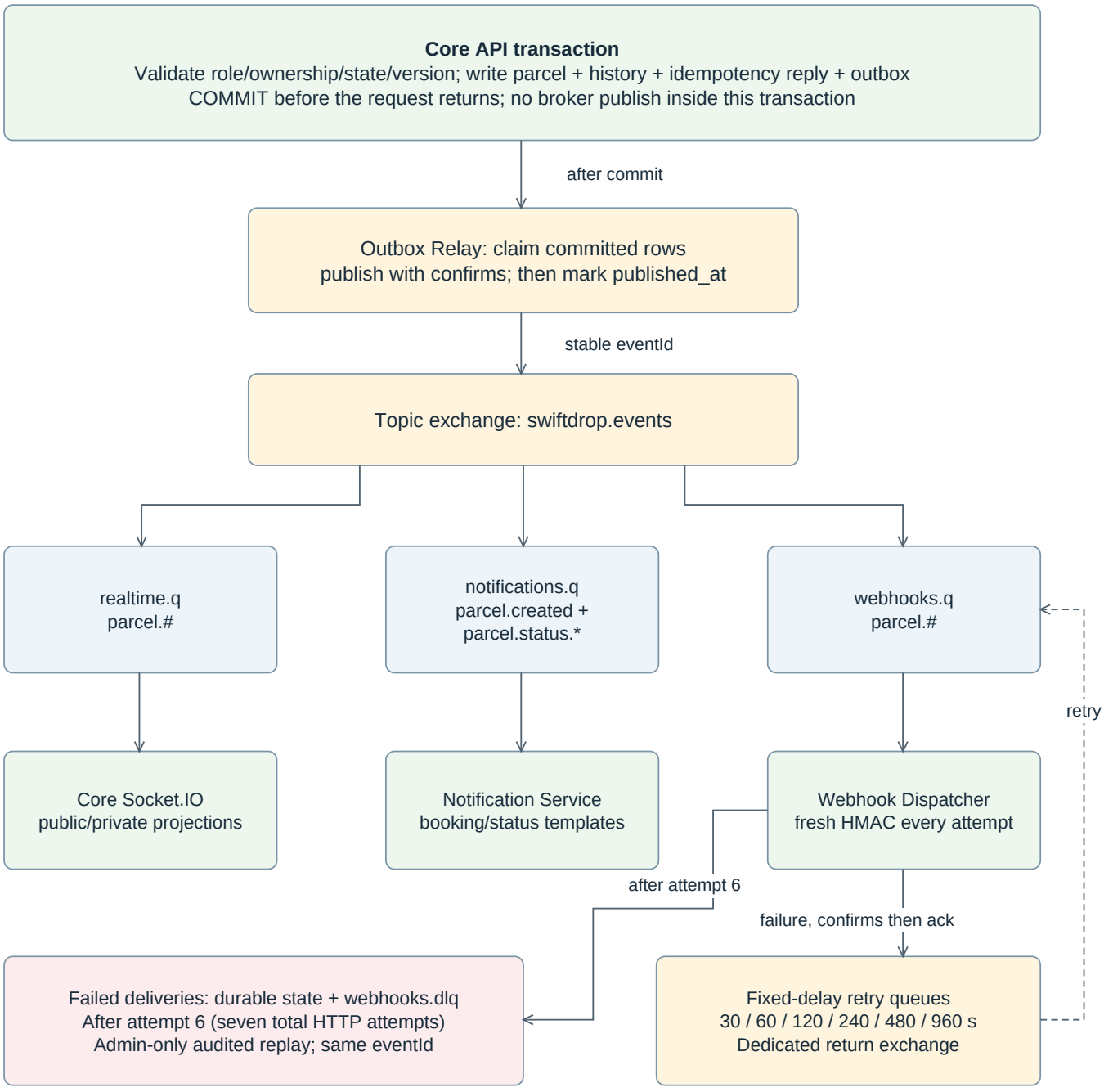



*Figure 8: Event-driven integration through RabbitMQ*

The relay polls about once per second. A publish followed by a crash before published_at may publish the same event again. Consumers persist progress and acknowledge only after completion or a confirmed durable retry/DLQ handoff. Booking confirmations have an explicit parcel.created binding.

Retry queues dead-letter only through the dedicated direct return exchange into webhooks.q, so retries never re-fire notifications or live tracking. Queue delay is approximate and can increase during outages; see Section 7.7.

## 6.3 Event contract

```
{
  "eventId": "6f1c2a9e-3b0d-4c7e-9a51-1d2f0c8b7e44",
  "type": "parcel.status.changed", "version": 1,
  "occurredAt": "2026-10-23T09:14:05Z", "sequence": 4,
  "data": {"parcelId": "p_1042", "trackingNo": "SD-7KQ2M9XA-P",
    "merchantId": "m_1042", "from": "IN_TRANSIT",
    "to": "OUT_FOR_DELIVERY"}
}
```

eventId is the stable outbox primary key; sequence increases with each parcel change; version is the envelope schema version. Realtime discards stale sequence numbers and reloads a snapshot on gaps/reconnect. Audit/notification/webhook consumers retain every unique eventId: a later sequence is not permission to discard an unsent earlier business event.

Events omit names, phone numbers, addresses, photos and free-text notes. A worker needing contact/configuration data calls an internal event-bound endpoint using its own restricted service JWT (Section 5.3). Payloads are validated on consumption; unknown schema versions are parked for inspection.

## 6.4 Queues and routing keys

*Table 9: RabbitMQ queues*

| Queue / binding | Consumer and failure handling |
|---|---|
| realtime.q / parcel.# | Core Socket.IO gateway; 60-second TTL. Lost/expired live updates are recovered from an authorised REST snapshot, not treated as a durable event history. |
| notifications.q / parcel.created and parcel.status.* | Notification Service; separate booking/status templates. At most three redeliveries after the initial attempt, then confirmed notifications.dlq handoff and operator visibility. |
| webhooks.q / parcel.# | Webhook Dispatcher; initialise attempt=0. Persist failures and schedule attempts 1..6 through the fixed-delay queues; no retry on the topic exchange. |
| webhooks.retry.{30,60,120,240,480,960}.q | No consumers. One fixed message TTL per queue; dedicated direct dead-letter return exchange routes only to webhooks.q. Configure quorum at-least-once dead-lettering and persistent storage. |
| webhooks.dlq / dedicated dead-letter route | Failed after attempt 6; retained 14 days, visible in delivery logs. Only Admin may replay; durable replay run retains original eventId. |

## 6.5 Reliability guarantees

A successful business transaction records parcel state, append-only history and an outbox event together. Publication and durable consumer queues provide at-least-once processing under the configured durability assumptions. A relay crash between confirm and marking published_at causes redelivery, so the same stable eventId is preserved.

Consumers claim a unique (consumer,eventId) record under a lease and persist progress. This suppresses tested duplicate scenarios, but external side effects have a crash window: an email or HTTP request can succeed before local completion is recorded. Use a provider idempotency key where supported; otherwise a duplicate email remains possible. Merchants deduplicate webhook eventId before business effects.

Use durable exchanges/queues, persistent messages, publisher confirms and mandatory routing checks. Retry and DLQ transfers are confirmed before acknowledging the source. Fixed-delay quorum queues must enable at-least-once dead-lettering with a compatible overflow policy and persistent volumes. Missing routes, unavailable targets, exhausted disk or permanent storage loss are operational failures; there is no unconditional no-loss guarantee.

### 6.6 Real-time tracking

Public clients join public:<trackingNo> after tracking-number validation and rate limiting. Every payload in that room is a public projection containing status, time, town and sequence only. Authenticated clients request private:<parcelId>; the server authorises ownership or current staff/courier access before membership and emits a separate private projection.

Never join a public socket to a private room or emit a full parcel into a public room. Reassignment/revocation invalidates access and removes private subscriptions; reconnect always reauthorises. Do not send recipient contact details through the broker or depend on a client-side filter to hide them.

Socket.IO falls back to long-polling when WebSockets are unavailable. Redis adapter broadcasts across API replicas. Multi-replica polling needs sticky sessions at the proxy. Clients deduplicate by sequence and fetch the latest authorised REST snapshot after reconnect or a gap, because realtime.q has a short TTL.

> Email, merchant webhooks and live tracking have different durability needs. Eventual delivery targets depend on the broker, workers and external endpoints being available; the report does not promise a fixed number of seconds during outages.

# 7 Key Flows

Sequences show interactions; state and activity diagrams show lifecycle and decisions. Dashed arrows denote replies or asynchronous messages; asynchronous broker transfers are labelled. The Outbox Relay is a separate participant after commit.

## 7.1 Creating a delivery

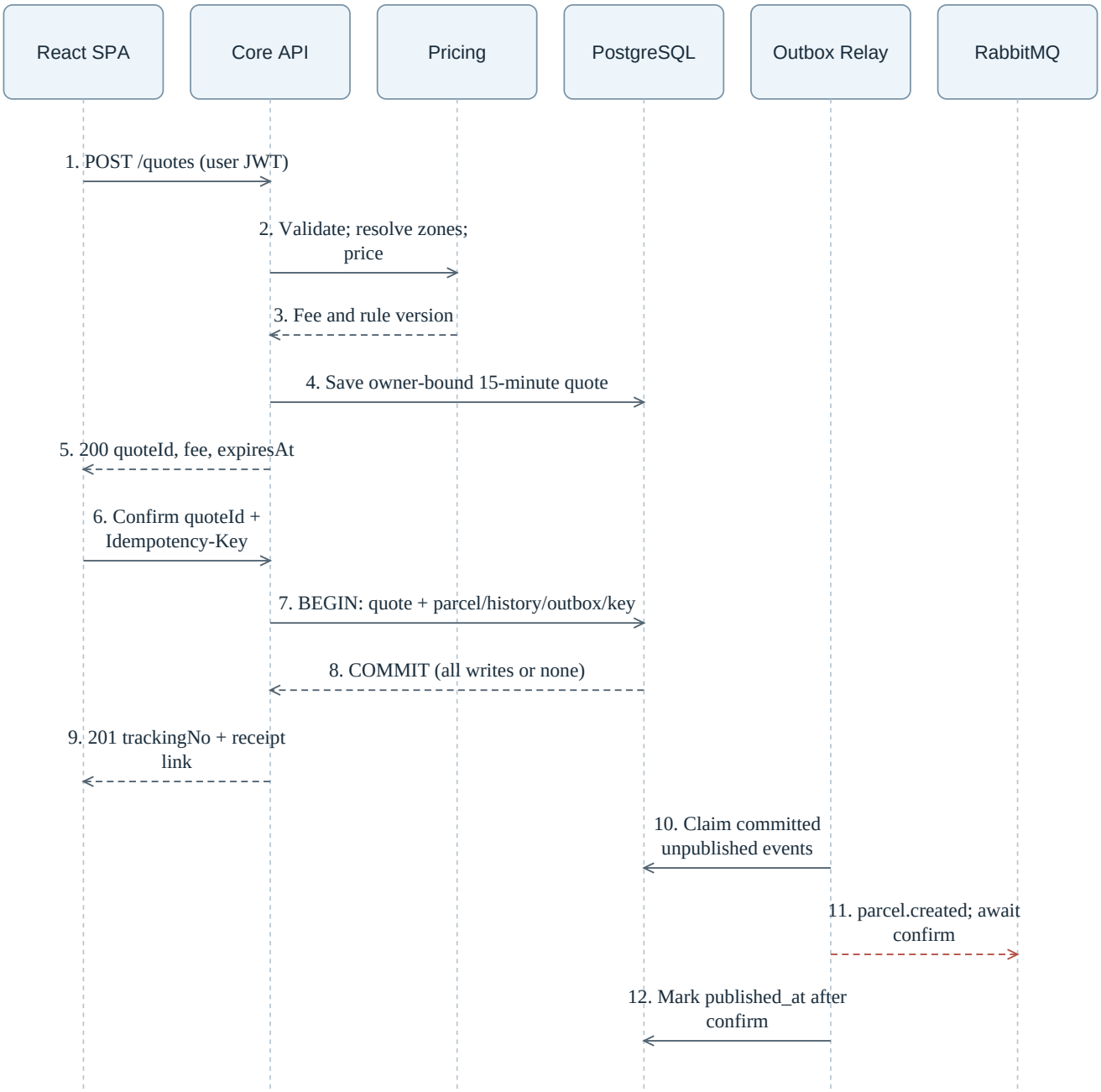



*Figure 9: Sequence: customer creates a delivery*

Two requests let the customer inspect the fee before booking. A quote is persisted and bound to the authenticated owner and exact inputs. Confirmation consumes it once inside the transaction. Same-key/body retries reuse the saved response; different-body reuse returns 409. The booking response never waits for email or broker delivery.

## 7.2 Merchant creates an order through the API

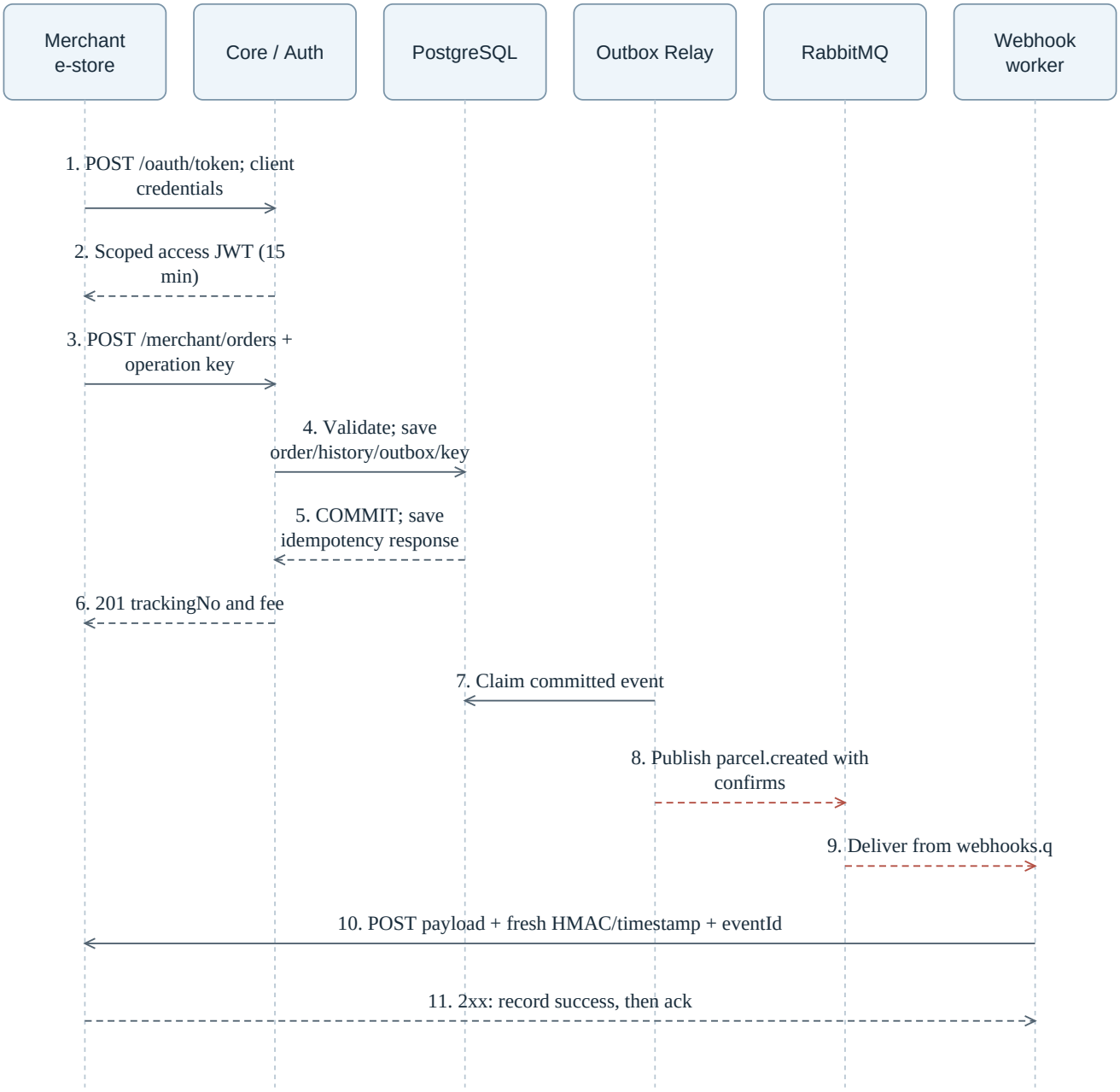



*Figure 10: Sequence: merchant API order and webhook*

Every order validates orders:write, current merchant approval and body fields before writing. Machine orders use merchant_id and may leave customer_id null. Merchant queries are scoped to that merchant. API creation uses an idempotency record to protect network retries and concurrent duplicate requests.

On timeout/non-2xx, the webhook worker persists the failure and confirms a durable retry handoff before acknowledging. Section 7.7 defines every attempt and replay branch. Each retry reloads current configuration and signs a new timestamp; the stable eventId lets the merchant suppress duplicate business effects.

## 7.3 Courier status update triggering notifications

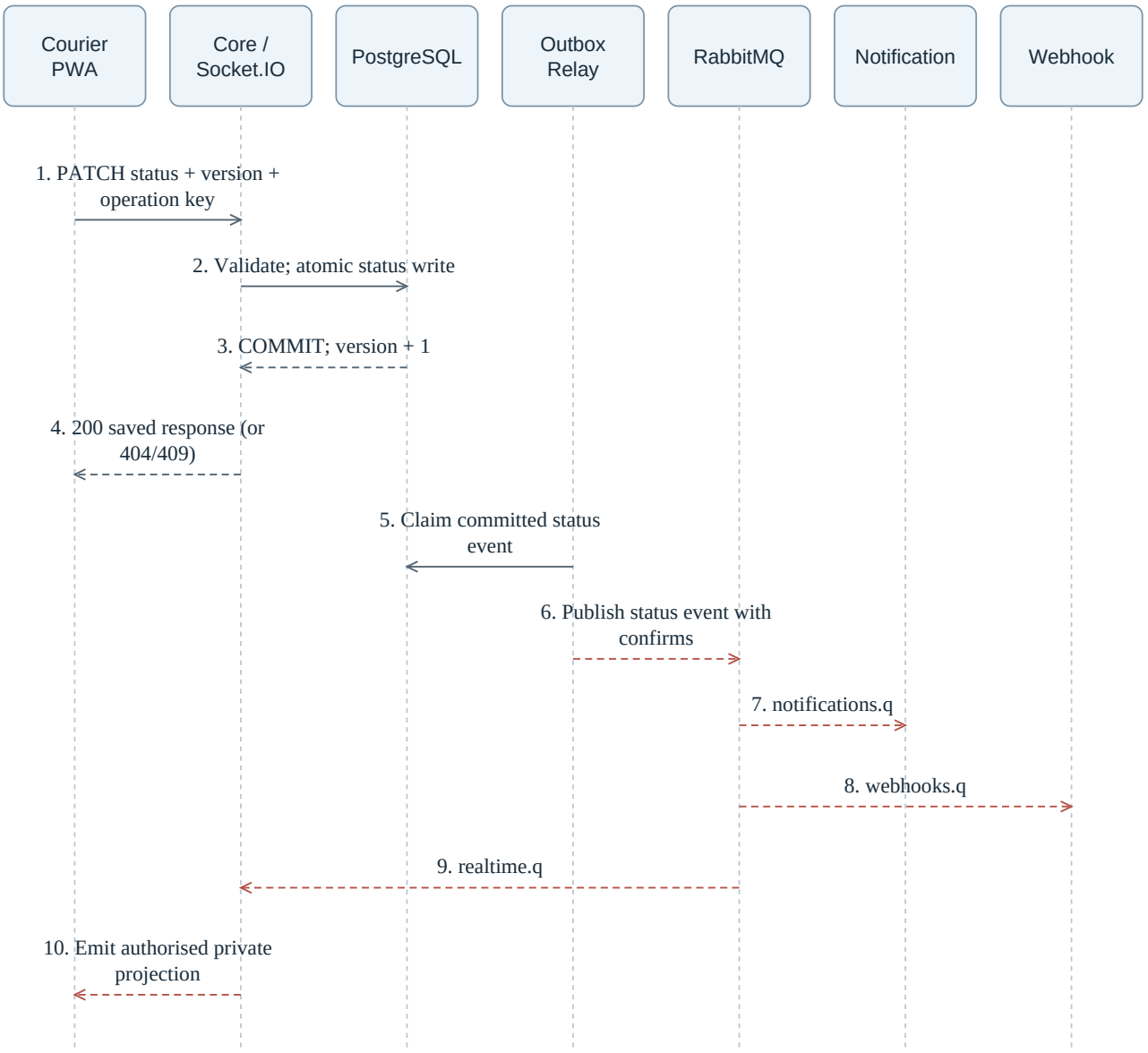



*Figure 11: Sequence: courier status update and independent fan-out*

Courier ownership misses return 404, role denials 403, and stale versions or illegal transitions 409. Validate recipient name for DELIVERED and fixed-list reason for FAILED. Private photo upload must complete and be authorised before a proof_object_key is accepted. Capacity-changing transitions use the shared courier-lock protocol.

Offline operations retain a stable key, expectedVersion and dependency order per parcel. On reconnect, refresh the authorised snapshot, retry pending uploads, then replay in order. Deduplicate accepted operations. Pause stale/conflicting operations for user review, and surface 404 after reassignment. Never fabricate success or blindly replace the current version. Queue loss on a lost device remains a limitation.

## 7.4 OAuth 2.0 login with Google

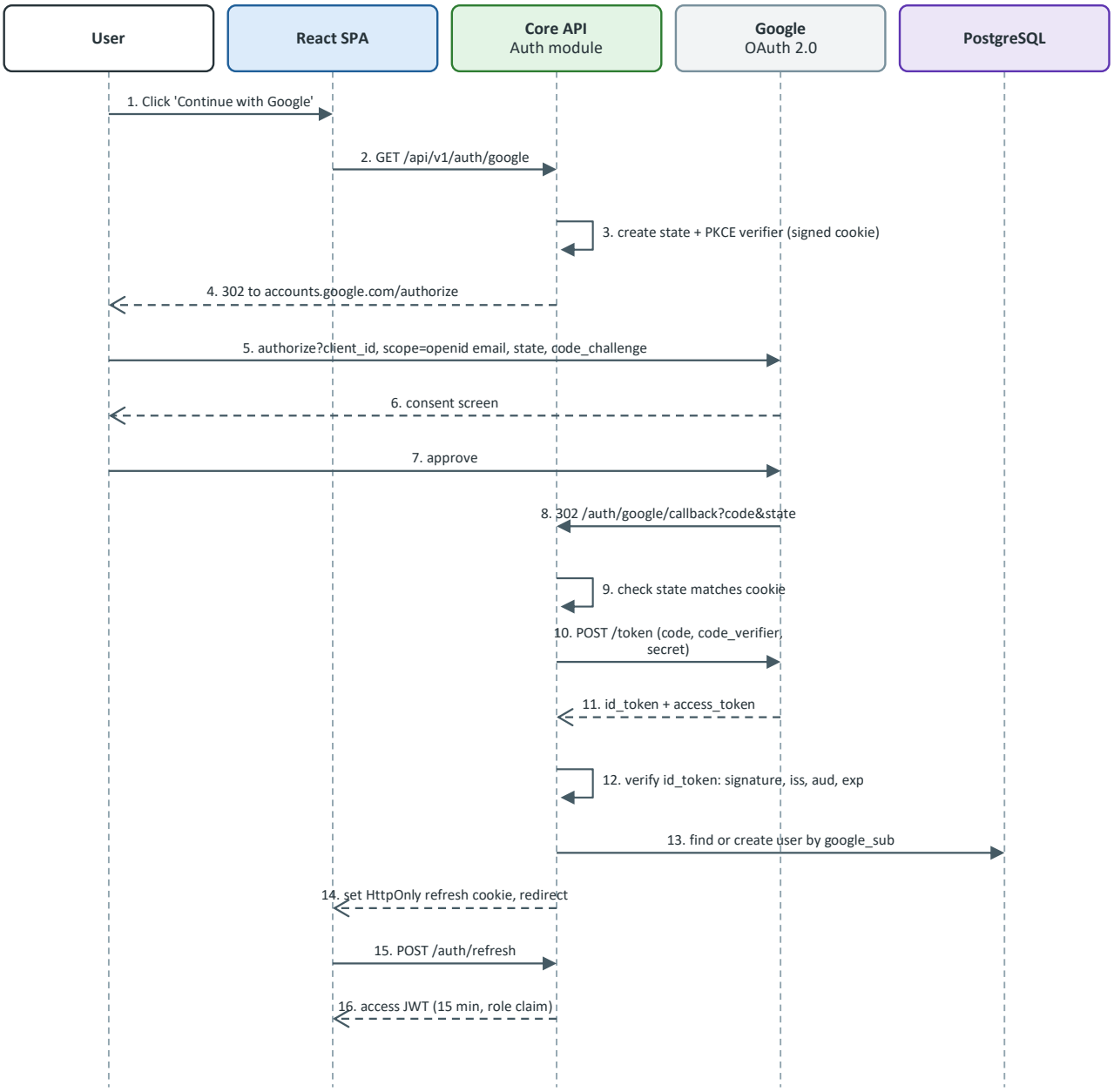



*Figure 12: Sequence diagram: Google sign-in with authorization code and PKCE*

We use the authorization code flow with PKCE, handled by the backend. The SPA never sees Google's tokens. The state value protects against cross-site request forgery on the callback, and the PKCE verifier ensures that an intercepted authorization code cannot be exchanged by anyone else. We verify the ID token's signature against Google's published keys and check the issuer, audience and expiry before trusting the email inside it. After that, Google is out of the picture: the user gets our own access JWT and a refresh token in an HttpOnly cookie, exactly as if they had logged in with a password.

## 7.5 Parcel lifecycle

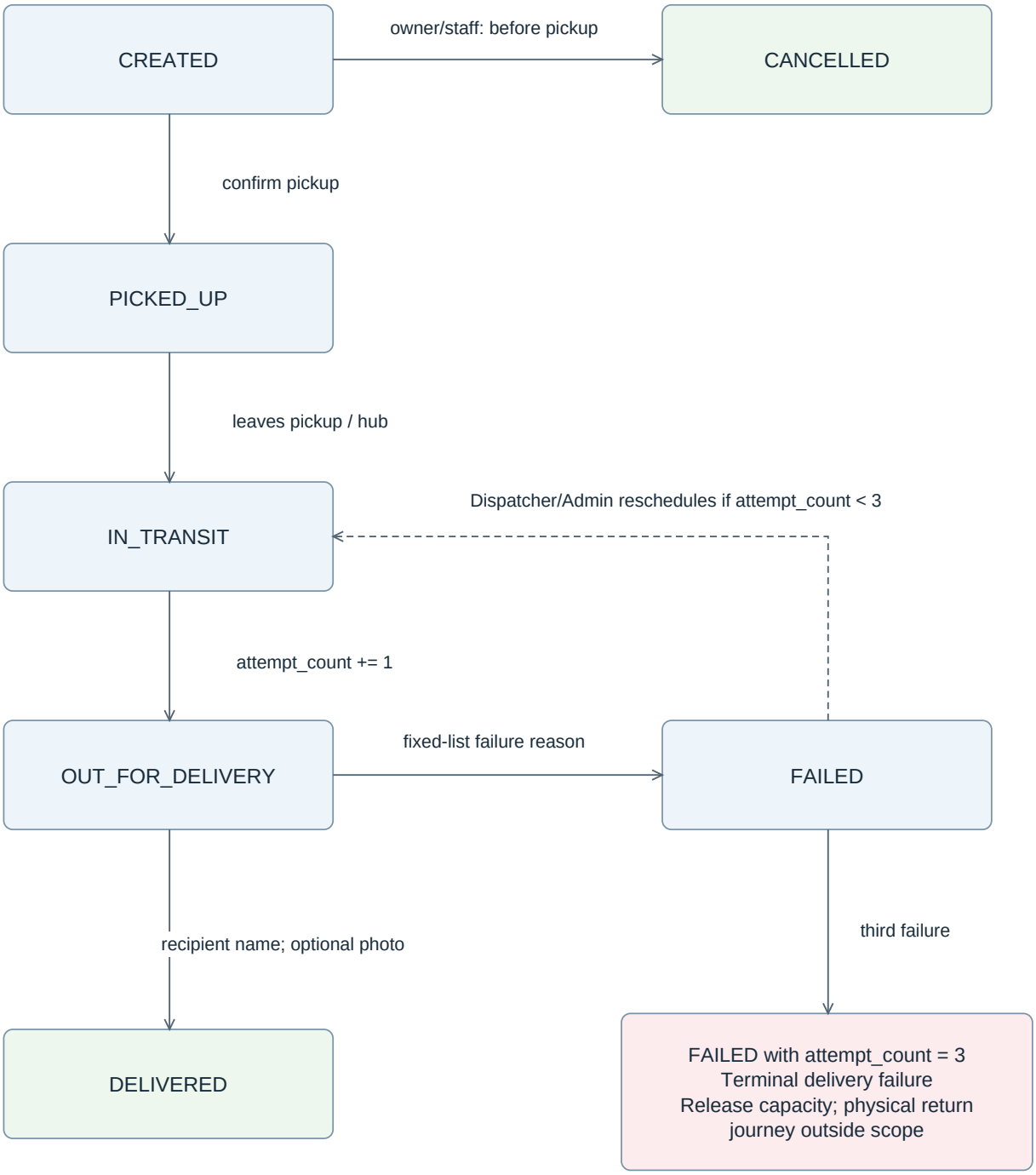



*Figure 13: State diagram: parcel lifecycle*

There are at most three delivery attempts: the first attempt plus two re-attempts. attempt_count begins at 0 and increments only on IN_TRANSIT to OUT_FOR_DELIVERY. A failed attempt 1 or 2 reserves capacity until rescheduled, reassigned or cancelled through an authorised path; cancellation itself is permitted only before pickup.

Only Dispatcher/Admin schedules a re-attempt. FAILED after attempt 3 is terminal and releases capacity. DELIVERED and CANCELLED are terminal. Assignment/reassignment changes courier and version, not lifecycle status.

## 7.6 Dispatcher assigns a parcel

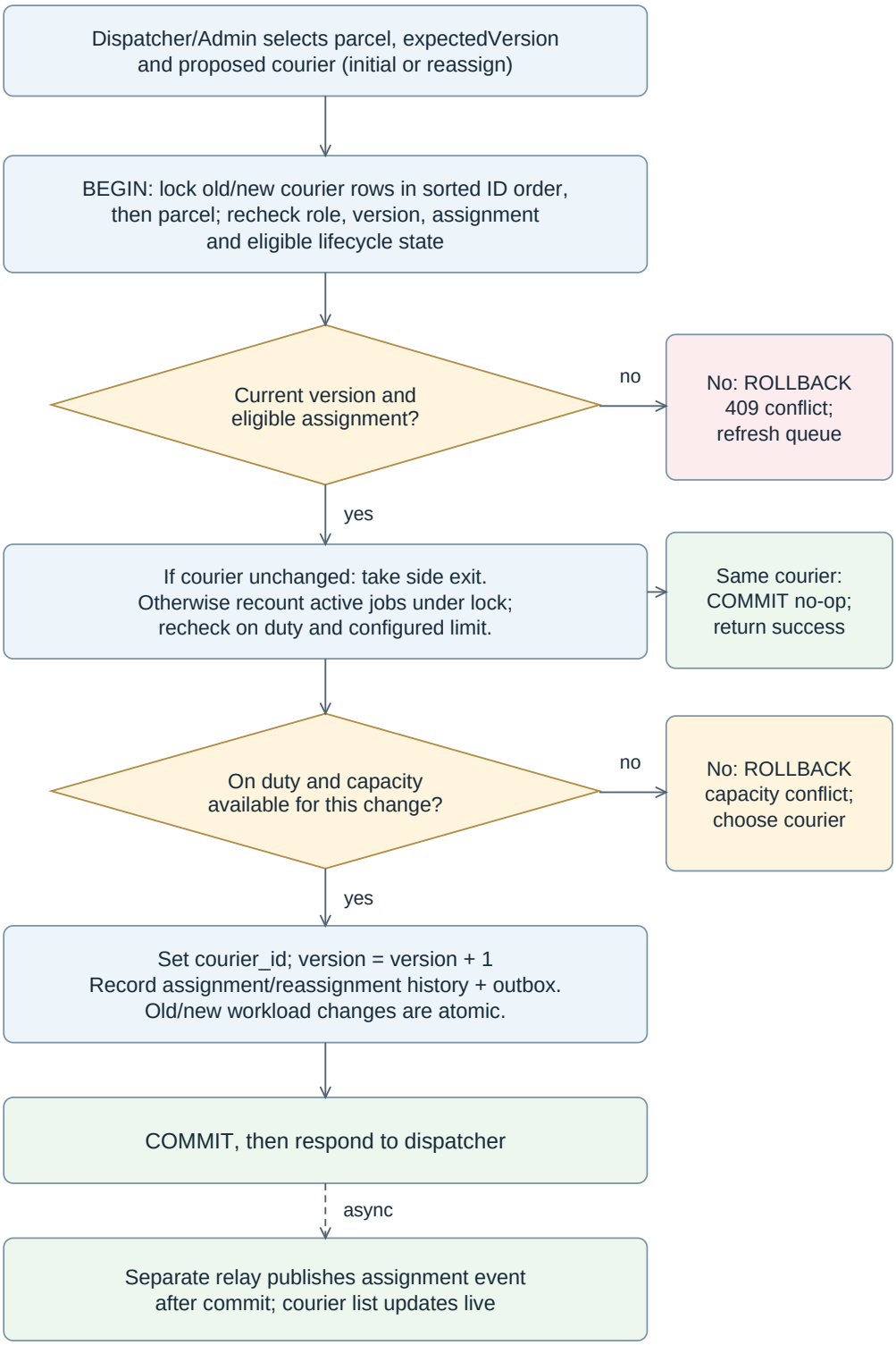



*Figure 14: Activity diagram: assignment and reassignment*

All assignment, duty and active-status changes use this lock order. Two different parcels cannot both claim the same final courier slot. If courier ownership changed before locks were acquired, restart from a fresh snapshot. Eligible reassignment is allowed for nonterminal parcels; first assignment requires an unassigned CREATED parcel.

## 7.7 Webhook delivery and retry

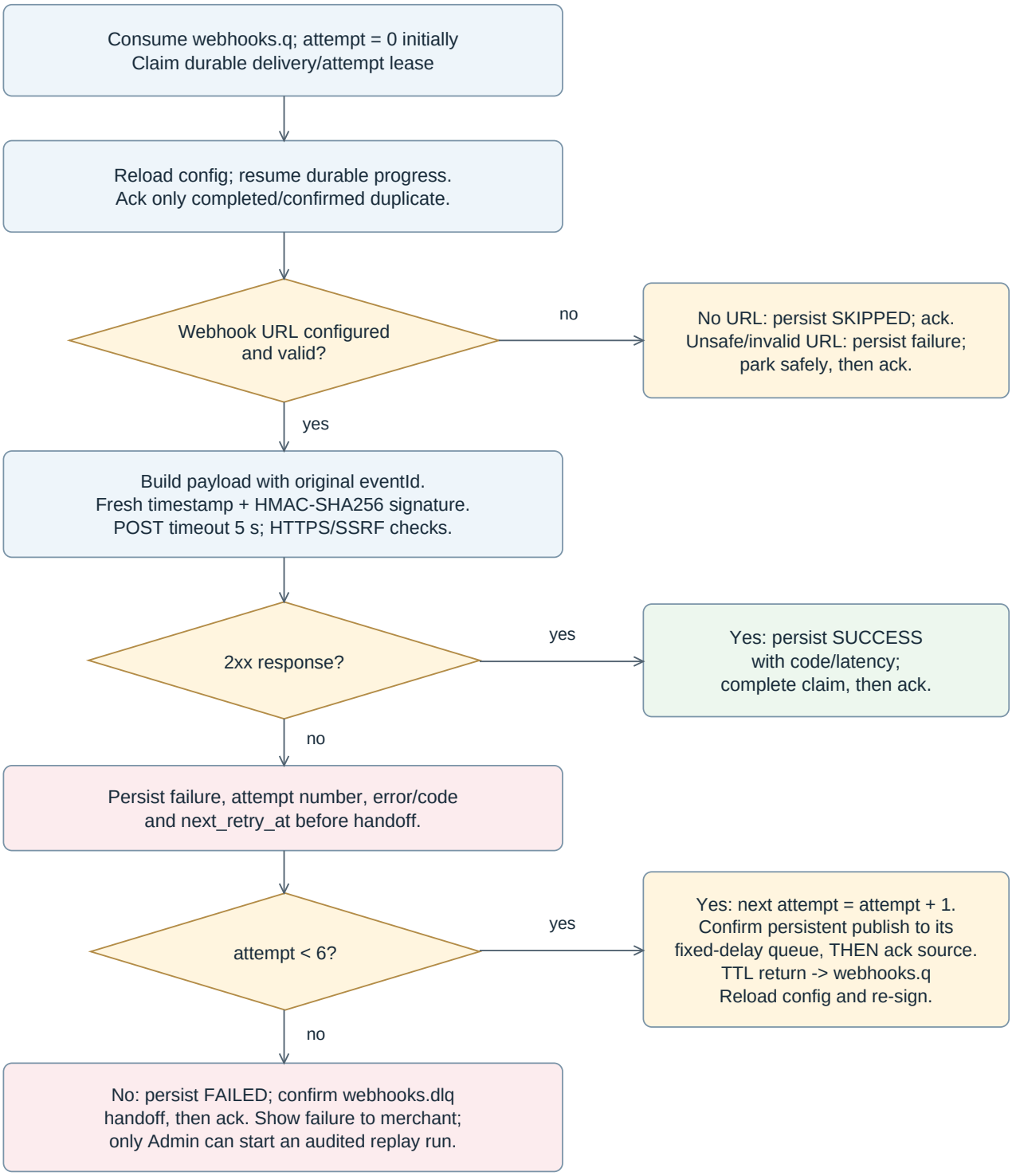



*Figure 15: Activity diagram: webhook delivery with retry*

One initial HTTP attempt (0) plus six retries (1..6): waits of 30, 60, 120, 240, 480 and 960 seconds; total 1,890 seconds (31.5 minutes), plus request/queue time. Re-delivery after an unconfirmed handoff resumes durable state; do not advance the counter or resend a completed attempt. Replays keep eventId and use a new audited run ID.

Fixed-delay queues avoid mixed-TTL head blocking. Use quorum at-least-once dead-lettering and persistent volumes; delay is approximate. If persistence or confirm fails, leave the source unacknowledged. An external success before local recording can still duplicate delivery.

# 8 Design Patterns and Principles

## 8.1 Patterns applied

*Table 10: Design patterns and where they are used*

| Pattern | Where | What it buys us |
|---|---|---|
| Repository | Every module's data access (ParcelRepository, PricingRuleRepository, ...) | Services depend on an interface, so unit tests use in-memory fakes and Prisma stays an implementation detail. |
| Strategy | Pricing: StandardPricing, ExpressPricing, MerchantContractPricing | New pricing schemes are new classes. The booking code does not change. |
| State | Parcel status transitions | One place defines which transitions are legal. Impossible jumps are rejected everywhere. |
| Observer / publish-subscribe | Status events to notification, webhook and realtime consumers | The Parcels module does not know who is listening. Adding a consumer touches no existing code. |
| Transactional outbox | Event publishing from Core API | Business change and event are saved atomically. |
| Adapter | EmailSender and SmsSender interfaces with SMTP and mock SMS adapters | Swapping the mock SMS for a real Sri Lankan SMS gateway later is a configuration change. |
| Factory | NotificationChannelFactory, PricingStrategyFactory | Choosing the right implementation is kept out of the business logic. |
| Chain of responsibility | Express middleware: authenticate, authorise, validate, handle | Each concern is one small function, reused across routes. |
| Dependency injection | Services receive repositories and clients through constructors | Easy to test and to replace dependencies per environment. |

The pricing strategy shows how small these pieces are in practice:

```
interface PricingStrategy {
  quote(input: QuoteInput, rule: PricingRule): Money;
}

class StandardPricing implements PricingStrategy {
  quote({ weightKg }: QuoteInput, rule: PricingRule): Money {
    const extraKg = Math.max(0, Math.ceil(weightKg) - 1);
    return Money.cents(rule.baseFeeCents + extraKg * rule.perKgCents);
  }
}

// e.g. Colombo -> Kandy, small parcel, 2 kg:
// base 45000 + 1 x 12000 = 57000 cents = LKR 570.00
```

## 8.2 Principles

• **Single responsibility.** Controllers translate HTTP, services hold business rules, repositories store data. The Notification Service sends messages; it does not decide which parcel transitions are legal.

• **Open/closed.** Pricing strategies, notification adapters and event consumers expose narrow contracts so new implementations can be added without rewriting booking orchestration. New requirements may still require deliberate interface changes.

• **Liskov substitution.** PricingStrategy and SmsSender implementations must satisfy the same documented contracts. Shared contract tests are planned for each implementation; no passing results are available yet.

• **Interface segregation.** Courier code depends on a narrow status-update interface rather than administrative methods it does not need. Worker lookup scopes expose only the data required for the event being processed.

• **Dependency inversion.** Domain services depend on interfaces. Infrastructure provides Prisma repositories, AMQP publishers and email/SMS adapters, injected through constructors.

• **DRY.** Keep validation schemas, allowed status transitions, money rules and event envelopes in one maintained contract package. Reuse the schema across API and UI, while always validating at the API boundary.

• **KISS and YAGNI.** The design avoids full microservices, a service mesh and event sourcing. They are unnecessary for the current team and scale. Independent workers address the specific need for outbound failure isolation.

The retained pattern table and pricing example show intended structure, not proof that classes or services already exist. Concrete module boundaries, contracts, migrations and CI checks remain implementation work.

# 9 Security Design

## 9.1 Authentication

• Hash passwords with bcrypt cost 12; reject inputs beyond its 72-byte UTF-8 limit rather than silently truncating. Do not log passwords or tokens.

• Use RS256 access JWTs valid for 15 minutes, containing user ID, role and jti plus standard issuer, audience and expiry claims. Do not include contact details.

• Use random refresh tokens stored hashed, with family and reuse-detection state. Send via HttpOnly, Secure, SameSite=Strict cookies; consume and rotate atomically. Reuse revokes the family. Enforce origin/CSRF checks for cookie-authenticated operations.

• Google uses backend-handled OpenID Connect code flow with PKCE and state, as shown in Section 7.4. Verify signature, issuer, audience and expiry; identify accounts by google_sub rather than silently merging by email.

• Rate-limit login/token endpoints by IP and account; check current account status for privileged operations. These controls are planned, not verified.

## 9.2 Authorisation

Authenticate, check role/scope, then scope the database operation to the owner or current courier. A foreign or nonexistent parcel returns the same 404 response. A known route forbidden to the caller's role/scope returns 403. Admin inherits Dispatcher operations; replay remains Admin-only. Machine clients remain limited by merchant ownership and token scope.

## 9.3 Merchant API and webhooks

• Show client secrets once and store only their hash. Support rotation/revocation; issue short-lived tokens with orders:read and/or orders:write scopes.

• Sign the exact HTTP body using HMAC-SHA256 with a fresh timestamp for every attempt. Merchant validates the timestamp (five-minute window), signature and eventId deduplication. Store signing secrets encrypted; never expose them in logs.

• Require HTTPS webhook URLs. Validate resolved IPv4/IPv6 destinations on every connection; reject private/loopback/link-local ranges and redirects. A development-only allowlist permits the mock receiver; it is empty in production. Apply timeouts, request-size bounds and per-client rate limits.

## 9.4 Personal data

• Validate all external input at API and event boundaries: phone format, positive bounded weight, known statuses/zones and length-limited notes. Use parameterised queries; never interpolate user input into SQL.

• Redact passwords, tokens, secrets, phone numbers and addresses in logs. Return generic errors with a correlation ID. Detailed server diagnostics also remain redacted.

• Public tracking shows status, time and town only. Full addresses/contact details require the owner, current assigned courier or authorised staff. Private proof objects and receipts are accessible only through authorised short-lived signed URLs; validate MIME type/size and ownership at upload/finalisation.

### 9.5 Secrets and configuration

The planned deployment reads secrets from environment/secret storage and commits only placeholder examples. Kubernetes, if implemented, uses ConfigMaps for non-sensitive settings and Secrets with restricted RBAC for credentials. Secret scanning is a required future CI check. No .env.example, deployment controls or working CI pipeline is claimed to exist in this document-only repository.

### 9.6 Threats and mitigations

*Table 11: Threat model summary (STRIDE)*

| Threat | Example | Required mitigation |
|---|---|---|
| Spoofing | Fake merchant integration | Scoped client-credentials tokens, hashed secrets, approval/revocation checks. |
| Tampering | Modified webhook body | HTTPS, exact-body HMAC, fresh timestamp and eventId validation. |
| Repudiation | Disputed delivery update | Append-only actor/time/status history, name and optional private proof. |
| Information disclosure | Guessed tracking number | Random IDs, rate limits, public projection and authorised private rooms. |
| Denial of service | Quote/webhook flood | Rate limits, bounded inputs, timeouts, bounded retries and pagination. |
| Elevation of privilege | Courier changes parcel ID | Role + ownership-scoped query, current assignment/version, negative tests. |

# 10 API Design

## 10.1 Conventions

Version business endpoints under /api/v1; /oauth/token is the OAuth endpoint. Use plural resources, cursor pagination and one problem-details error format. OpenAPI generation from Zod and /api/docs are planned. Creation and courier mutations use a principal-scoped Idempotency-Key with request hash; state changes require expectedVersion. All paths below are full paths.

## 10.2 Main endpoints

*Table 12: Key REST endpoints (summary; not exhaustive)*

| Method and path | Role / purpose |
|---|---|
| POST /api/v1/auth/register<br>POST /api/v1/auth/login | Public: customer sign-up and login. |
| GET /api/v1/auth/google<br>GET /api/v1/auth/google/callback | Public: Google code flow. |
| POST /api/v1/auth/refresh | Refresh cookie: rotate family token, issue access JWT. |
| POST /api/v1/quotes | Customer, Merchant, Dispatcher, Admin: fee quote. |
| POST /api/v1/parcels | Customer, Dispatcher, Admin: create from quote. Merchant machine creation uses merchant/orders. |
| GET /api/v1/parcels<br>GET /api/v1/parcels/{id}<br>GET /api/v1/parcels/{id}/receipt | Customer/Merchant: own; Courier: assigned detail; Dispatcher/Admin: authorised operational view. Receipts only owner or staff. |
| GET /api/v1/track/{trackingNo} | Public: status, time and town only. |
| POST /oauth/token | Approved Merchant client: client-credentials token. |
| POST /api/v1/merchant/register<br>PUT /api/v1/merchant/webhook | Public registration (pending); Merchant owns webhook settings; Admin may manage. |
| POST /api/v1/merchant/orders<br>GET /api/v1/merchant/orders | Machine token requires orders:write/read; approved merchant, owner-scoped queries. Dashboard Merchant JWT permits own read. |
| GET /api/v1/courier/parcels?date=today<br>PATCH /api/v1/courier/parcels/{id}/status<br>POST /api/v1/courier/parcels/{id}/proof | Courier: assigned parcels only; expectedVersion + operation key for update; private proof upload/finalise. |

## 10.2 Main endpoints (continued)

| Method and path | Role / purpose |
|---|---|
| GET /api/v1/admin/parcels?status=CREATED&assigned=false | Dispatcher, Admin: unassigned CREATED queue; assigned=false derives from courier_id IS NULL, not a lifecycle status. |
| POST /api/v1/admin/parcels/{id}/assignment<br>GET /api/v1/admin/couriers/workload | Dispatcher, Admin: assign/reassign with expectedVersion; view active workload. |
| PATCH /api/v1/admin/parcels/{id}/status<br>POST /api/v1/admin/parcels/{id}/reattempt | Dispatcher, Admin: legal operational status update; schedule re-attempt only after FAILED with attempt_count < 3. |
| POST /api/v1/parcels/{id}/cancel | Customer/Merchant owner, Dispatcher, Admin: cancel only before pickup; version and shared capacity locks required. Merchant machine needs orders:write. |
| PATCH /api/v1/admin/merchants/{id}<br>POST/PATCH /api/v1/admin/users<br>POST/PATCH /api/v1/admin/couriers | Admin: approve merchants, provision/suspend users and couriers. |
| CRUD /api/v1/admin/zones<br>CRUD /api/v1/admin/pricing-rules | Admin: zones and pricing. |
| GET /api/v1/admin/reports/daily | Dispatcher, Admin: daily reports and CSV export. |
| POST /api/v1/admin/webhook-deliveries/{id}/replay | Admin only: audited replay run; preserve original eventId; no new notification/tracking event. |

### 10.3 Error format

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://api.swiftdrop.lk/errors/invalid-transition",
  "title": "Status change not allowed", "status": 409,
  "detail": "A parcel in CREATED cannot move to DELIVERED.",
  "correlationId": "c0a8012e-5f3b"
}
```

401: missing/invalid authentication. 403: known role/scope denial. 404: ownership-scoped miss. 409: stale version, illegal transition or conflicting operation key. Errors do not disclose foreign parcel data.

# 11 Testing Approach

This is a verification plan. No application source, schema, automated tests or CI results were supplied; passing behavior and coverage are not claimed. The report correction was checked separately for content consistency and PDF layout.

*Table 13: Planned test levels*

| Level / tools | Meaningful scenarios |
|---|---|
| Unit: Jest | Integer pricing and quote expiry; legal transitions; proof/name and failure reason rules; first plus two re-attempts; public/private payload projection. |
| Integration: Supertest, Testcontainers (PostgreSQL, RabbitMQ) | Atomic state/history/outbox/idempotency writes and rollback; relay crash after confirm; duplicate operation keys; refreshed token reuse; private proof authorisation. |
| Authorisation: tokens for every role and machine scope | Foreign courier/customer/merchant lookup/update returns 404; wrong role/scope returns 403; admin inherits dispatch; replay denied to Dispatcher; private socket room reauthorises after reassignment. |
| Concurrency: real PostgreSQL transactions | Same parcel assigned twice; two parcels compete for one courier slot; reassignment locks old/new couriers; duty changes, delivery, cancellation and capacity changes race without exceeding the limit. |
| Events: real broker, mock SMTP/HTTP | Booking reaches parcel.created template; tested duplicate suppression; acknowledge only after durable completion/handoff. Crash after external send is tested/documented as a possible duplicate without provider idempotency. |
| Retry / replay / failure | No URL, unsafe URL, 2xx, timeout, non-2xx; attempts 0..6 and 31.5-minute minimum waits; fresh signatures; failed confirms leave source unacked; restart recovery, DLQ retention and admin-only replay. |
| Frontend: Vitest, React Testing Library | Validation and duplicate-submit prevention; loading/empty/error/success states; keyboard/focus and labels; reduced motion; narrow/mobile layout; offline upload/replay/conflict handling. |
| Realtime / privacy | Public room contains only approved fields; unauthorised joins fail; Redis multi-replica fan-out; snapshot after expiry, gap and reconnect; polling/sticky-session behavior. |

Target at least 80% service/domain line coverage once code exists, using Jest/Istanbul HTML and lcov output. Coverage alone does not establish concurrency safety or secure ownership. Run deterministic tests and user-led manual checks; do not use Selenium or browser automation for testing.

# 12 Bonus Features (optional stretch)

## 12.1 Kubernetes deployment

After the baseline is demoable, a Helm chart may deploy stateless services as Deployments and stateful dependencies as StatefulSets with PVCs. Ingress replaces Nginx routing/TLS; ConfigMaps and restricted Secrets hold configuration. Readiness/liveness probes and scaling values need deployment-specific design and tests. No working cluster or chart is claimed.

## 12.2 AI delivery assistant

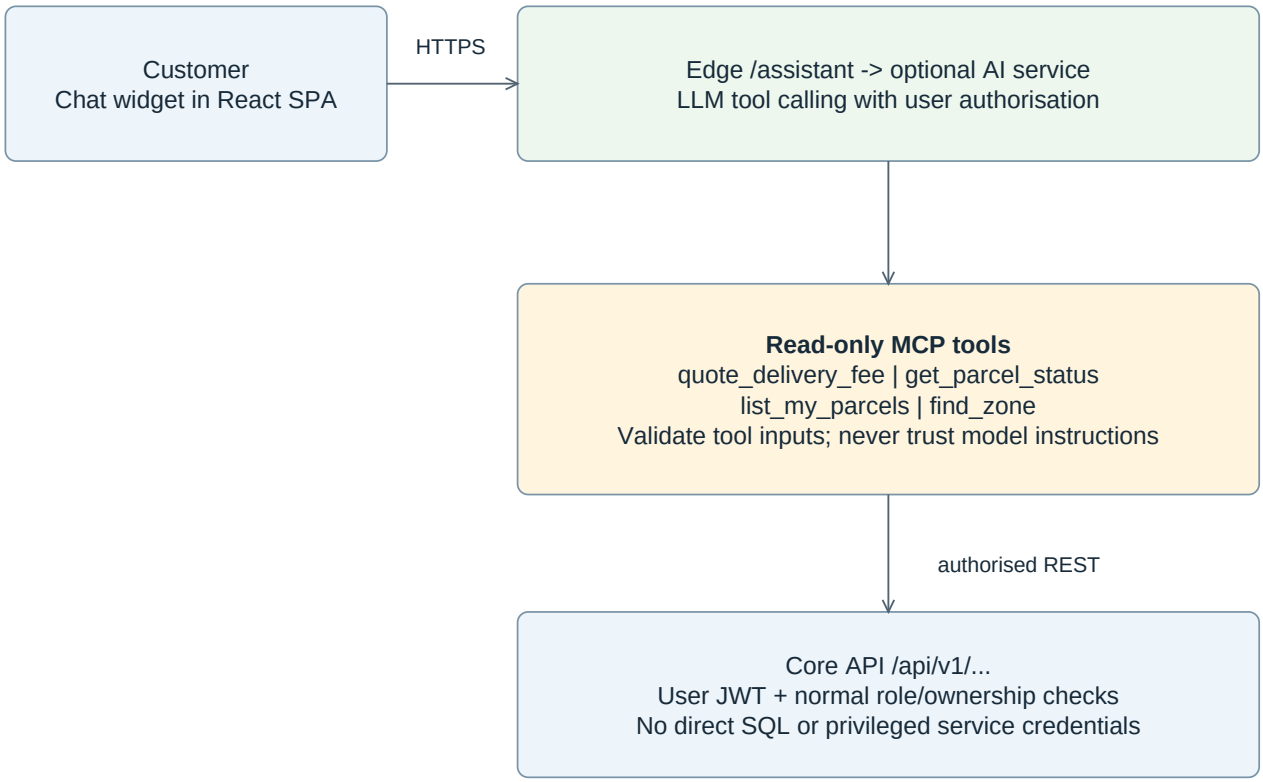



*Figure 16: Optional AI assistant through authenticated API tools*

The assistant can answer tracking and fee questions by calling real APIs rather than guessing. Tools are read-only: they cannot create, update or cancel parcels. A quote calculation may persist a short-lived quote but cannot create a delivery. The assistant can access only what the user can access.

Keep provider keys on the service, bound request/tool budgets, and minimise personal data sent to the model. Tool failures and unavailable providers produce an honest unavailable response. Optional AI traffic follows the same explicit edge route in both logical and deployment views.

# 13 Conclusion

The design rests on complementary decisions. A modular Core API keeps related business rules and database transactions together. A transactional outbox records events with business changes, then independent RabbitMQ consumers handle slow outbound work after commit. Role/scope checks and ownership-scoped queries provide explicit access boundaries.

The outbox promises atomic recording and at-least-once publication under stated durability assumptions. Stable event IDs, durable progress and receiver/provider idempotency reduce duplicates; external-send crash windows remain explicit. Courier capacity is protected across parcels by shared row locks, while version checks prevent stale parcel writes.

Public tracking has a deliberately small projection and separate socket rooms. Proof photos stay private. Offline replay validates current ownership, versions and upload state rather than silently replacing newer server data. These rules need real integration and concurrency tests before they can be described as working controls.

Docker Compose is the baseline deployment target. Kubernetes and the AI assistant are stretch features, not prerequisites. Online payments, route optimisation, physical return tracking and a future Parcels service split remain outside the current baseline.

## Validation and remaining limitations

• This is a corrected design report. There is no application implementation, schema, deployment configuration or test evidence in the supplied working tree.

• The original assignment brief/marking rubric was not supplied. Report coverage can be checked against the audit, but compliance with unseen marking requirements cannot be certified.

• The contribution sheet retains ten roles and blank member identities for the team to complete. No invented names, results or approval decisions have been added.

• The revised report source and review notes are identified in Appendix B. The product Markdown, delivery plan and authentication plan now share this corrected baseline. The original input PDFs remain preserved; no implementation or assessment result is implied.

# Appendix A: Contribution Sheet

Fill in names and registration numbers. The suggested areas match the module boundaries in Section 5, so each member owns a part they can explain during the demo.

*Table 14: Contribution sheet*

| # | Member name | Reg. no. | Main responsibility | Report sections |
|---|---|---|---|---|
| 1 | | | Team lead, architecture, integration design | 1, 5, 13 |
| 2 | | | Auth module: JWT, Google OAuth, RBAC | 2, 7.4, 9.1-9.2 |
| 3 | | | Parcels module, state machine, tracking API | 4, 7.5 |
| 4 | | | Pricing strategies, zones, quotes | 7.1, 8 |
| 5 | | | Dispatch module, courier workload | 3.4, 7.6 |
| 6 | | | Merchant API, client credentials, webhooks | 3.2, 7.2, 7.7, 9.3 |
| 7 | | | RabbitMQ, outbox relay, notification service | 6 |
| 8 | | | Frontend: customer portal, public tracking, real-time | 3.1, 6.6 |
| 9 | | | Frontend: merchant, courier PWA, admin console | 3.3, 10 |
| 10 | | | DevOps: Docker, Kubernetes, CI, testing and coverage | 5.5, 11, 12 |

# Appendix B: Editing and Review Notes

This corrected PDF is generated from docs/report/build_report.py. Diagrams on revised pages are vector drawing instructions in that file; body text, tables, captions and page references are maintained there too. It does not require an unavailable Word source or Word field updates.

The original merchant use-case page (9), Google login sequence (27), pattern table/pricing example (31) and contribution sheet (40) are imported unchanged from the supplied PDF because their existing content works. The original file is an input dependency. Regenerate, re-extract text and render every page after edits; do not patch the final PDF as a separate source.

SWIFTDROP_DESIGN_DOCUMENTATION.md is the reconciled product contract; plan.md controls delivery and ownership; docs/auth.md refines authentication and labelled extensions. They share this corrected baseline, including the resolved plan-only D3 priorities, ownership reference and sprint wording. Mermaid diagrams and this PDF use separate drawing formats; update affected representations together.

## Correction scope

Reviewed all audit groups A1-A4, B1-B7, C1-C4 and D1-D3. The report and source plans align on permissions, routing, delivery guarantees, proof, locks, retries, attempts, conceptual persistence, privacy and optional scope. Authentication phases retain the ten-person sprint order; mailbox verification is an optional extension and does not gate baseline booking/onboarding. Original working pages, group 23 and the contribution sheet are retained.

## Primary references for technical corrections

OMG UML extend notation
https://issues.omg.org/issues/UML25-428

RabbitMQ message TTL
https://www.rabbitmq.com/docs/ttl

RabbitMQ dead-letter safety
https://www.rabbitmq.com/docs/dlx

Node.js supported release status
https://nodejs.org/en/about/previous-releases

References reviewed on 29 October 2026. Runtime/image patches and broker policies must be validated when implementation begins. PDF inspection validates the report, not application behavior.